

Machine Learning: The No-Nonsense (But Fun) Guide

Day 28 – Docker: The "It Works on My Machine" Killer

Docker: Package Once, Run Anywhere

Introduction

One of the biggest challenges in software development and Machine Learning is ensuring that an application behaves the same way across different computers.

Imagine spending weeks building an ML model that works perfectly on your laptop. You send it to a teammate, and suddenly nothing works.

Typical responses include:

- "It doesn't work on my machine."
- "I'm using a different Python version."
- "That package isn't installed."
- "Windows behaves differently from Linux."

Docker solves this problem by packaging your application along with everything it needs to run—including the operating system libraries, Python version, dependencies, and source code—into a portable unit called a container.

Instead of sharing installation instructions, you simply share a Docker image.

What is Docker?

Docker is an open-source containerization platform that allows developers to package applications and all their dependencies into lightweight, portable containers.

A Docker container runs consistently across different environments, whether it's your laptop, a cloud server, or a production cluster.

Definition

Docker is a containerization platform that packages an application, its runtime, libraries, dependencies, and configurations into an isolated container, ensuring consistent execution across different environments.

Why Docker?

Without Docker:

- Install Python manually
- Install libraries individually
- Resolve dependency conflicts
- Different operating systems cause compatibility issues
- Difficult to reproduce environments

With Docker:

- One image contains everything
 - Same environment everywhere
 - No dependency conflicts
 - Fast deployment
 - Highly portable
 - Easy collaboration
-

Traditional Deployment vs Docker

Traditional Approach	Docker Approach
Install Python manually	Python included in image
Install dependencies separately	Dependencies packaged together
Environment differences	Same environment everywhere
Dependency conflicts	Isolated containers
Difficult setup	One command deployment

Docker Components

Docker consists of several important components that work together.

1. Docker Engine

The core software responsible for building, running, and managing containers.

2. Docker Client

The command-line interface (CLI) that users interact with.

Example:

docker build

docker run

docker ps

3. Docker Daemon

Runs in the background and performs all Docker operations.

Responsibilities include:

- Building images
- Running containers
- Managing networks
- Managing volumes

4. Docker Registry

A repository where Docker images are stored.

Popular registries include:

- Docker Hub
- GitHub Container Registry
- Amazon ECR
- Google Artifact Registry

5. Docker Image - IMP

An image is a read-only blueprint used to create containers.

It contains:

- Application code
- Python version
- Libraries
- Dependencies
- Environment variables
- Configuration

Analogy

Image = Recipe

You cannot cook from ingredients alone—you follow a recipe.

Similarly, Docker uses an image to create containers.

6. Docker Container

A container is a running instance of an image.

It contains:

- Running application
- Memory allocation
- Process isolation
- Network
- Temporary writable layer

Analogy

Container = Finished Meal

The recipe (image) creates the actual meal (container).

7. Dockerfile

A Dockerfile is a text file containing instructions for building an image.

Example instructions include:

- Select base image
 - Install libraries
 - Copy files
 - Configure ports
 - Define startup command
-

8. Docker Volume

Volumes provide persistent storage outside containers.

Used for:

- Saving datasets
- Database storage
- Model files
- Logs

Without volumes, data is lost when a container is removed.

9. Docker Network

Allows multiple containers to communicate securely.

Example:

- ML API
- Redis Cache
- PostgreSQL Database

can communicate over the same Docker network.

Docker Architecture

Developer

|



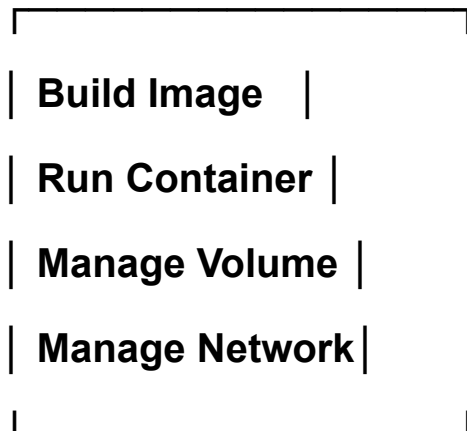
Docker CLI

|



Docker Daemon

|



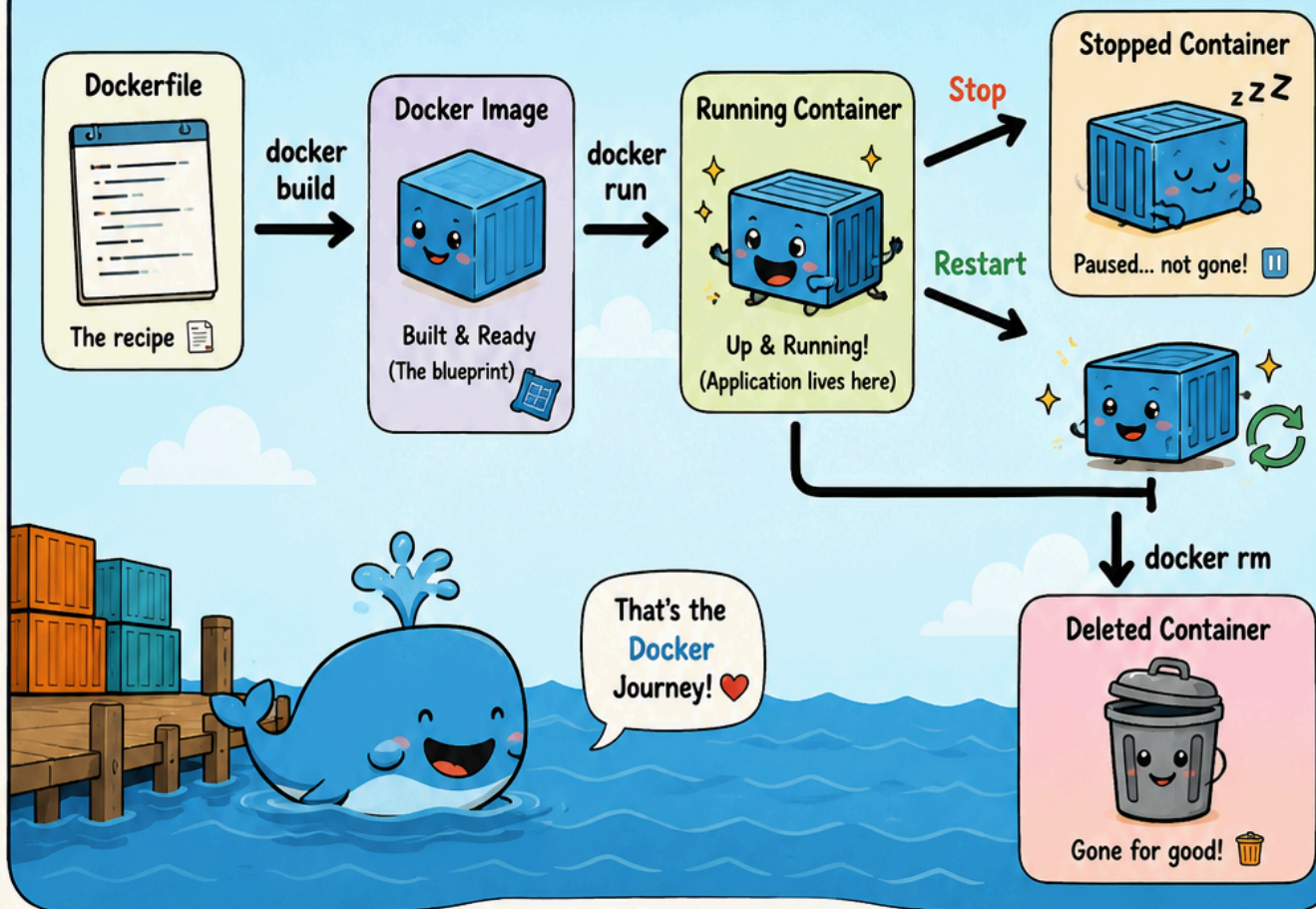
Docker Lifecycle - in fun wayy !!



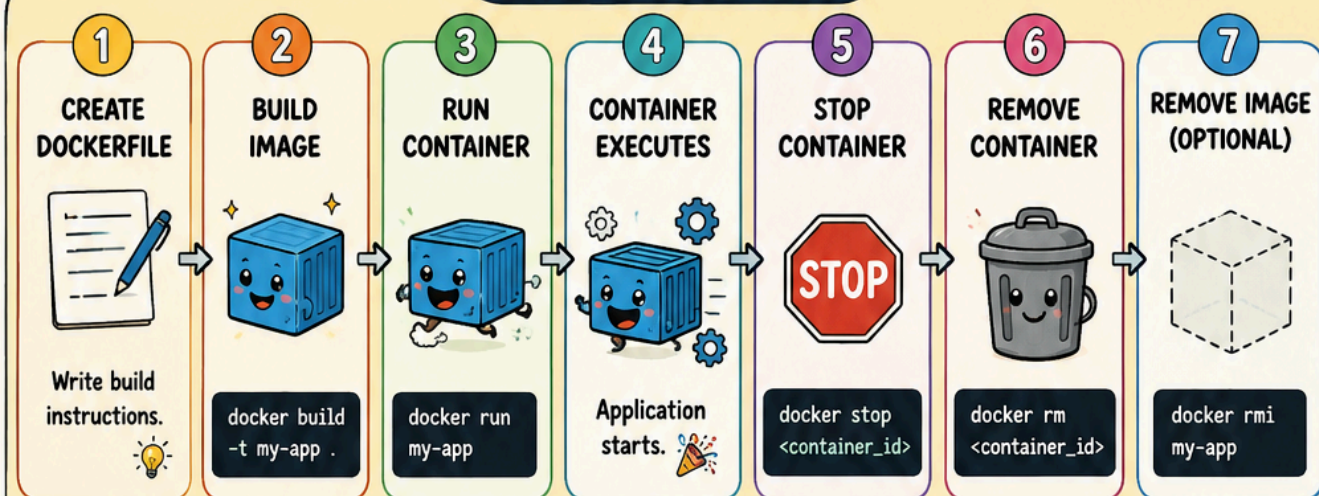
DOCKER LIFECYCLE

FROM CODE TO CONTAINER AND BEYOND!

LIFECYCLE FLOW



LIFECYCLE STEPS



Build it. Run it. Stop it. Clean it.
That's Docker! ❤️



Simple,
Powerful,
Awesome! 😎

Key Docker Concepts in short

Component	Analogy	Description
Image	Recipe	Blueprint used to create containers
Container	Running application	Executable instance of an image
Dockerfile	Cooking instructions	Defines how to build an image
Volume	External hard drive	Persistent storage
Registry	App Store	Stores and shares Docker images
Network	Road system	Enables communication between containers

Why Machine Learning Engineers Need Docker

Docker solves several common ML deployment challenges.

Problem	Docker Solution
Different Python versions	Pin exact version in image
Library conflicts	Isolated containers
Difficult deployment	Portable container
Team collaboration	Share images instead of setup guides
Reproducibility	Identical environments
Scaling	Launch many identical containers

Dockerfile for an ML Project

FROM python:3.9-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
gcc \

**libgomp1 **

&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

Example requirements.txt

numpy==1.24.3

pandas==2.0.3

scikit-learn==1.3.0

tensorflow==2.13.0

flask==2.3.2

Common Docker Commands

Command	Description
<code>docker build -t app .</code>	Build image
<code>docker run app</code>	Run container
<code>docker run -d app</code>	Run in background
<code>docker run -p 5000:5000 app</code>	Map ports
<code>docker run -v host:container app</code>	Mount volume
<code>docker ps</code>	List running containers
<code>docker images</code>	List images
<code>docker stop <id></code>	Stop container

<code>docker rm <id></code>	Remove container
<code>docker rmi image</code>	Remove image
<code>docker exec -it <id> bash</code>	Enter running container
<code>docker logs <id></code>	View logs

Docker Compose for ML Pipelines

Docker Compose allows multiple services to be managed together.

Typical services include:

- ML Prediction API
- Redis Cache
- PostgreSQL Database
- Jupyter Notebook
- Monitoring tools

Run all services together using:

`docker-compose up`

Docker vs Virtual Machines - imp

Feature	Docker Container	Virtual Machine
Size	MBs	GBs
Startup Time	Seconds	Minutes
Performance	Near Native	Slower
Isolation	Process Level	Full OS Level
Resource Usage	Lightweight	Heavy
Portability	Excellent	Good

Docker Use Cases

Docker is widely used across software development, DevOps, Data Engineering, and Machine Learning.

Machine Learning

- Model deployment
- Model serving APIs
- Reproducible training environments
- Experiment tracking
- Feature engineering pipelines

Data Engineering

- ETL pipelines
- Apache Spark clusters
- Airflow orchestration
- Kafka streaming applications

Software Development

- Microservices
- Web applications
- REST APIs
- Backend services

Cloud Deployment

- Kubernetes
- AWS ECS
- Azure Container Apps
- Google Cloud Run

Continuous Integration & Deployment (CI/CD)

- Automated testing
- Automated builds
- Deployment pipelines
- Version-controlled environments

Research & Education

- Sharing reproducible experiments
- Classroom labs
- Data science workshops
- Collaborative development

Interview Questions

1. What is Docker and why is it useful?

- 2. What is the difference between an image and a container?**
 - 3. What is a Dockerfile?**
 - 4. How does Docker ensure reproducibility?**
 - 5. What is Docker Compose?**
 - 6. What are Docker volumes?**
 - 7. How are Docker containers different from Virtual Machines?**
 - 8. Explain the Docker lifecycle.**
 - 9. What are common use cases of Docker in Machine Learning?**
 - 10. How would you deploy an ML model using Docker?**
-

Happy learning !!!